

Ex. No: 4 CPU Scheduling Algorithms

1. First Come First Serve (FCFS):

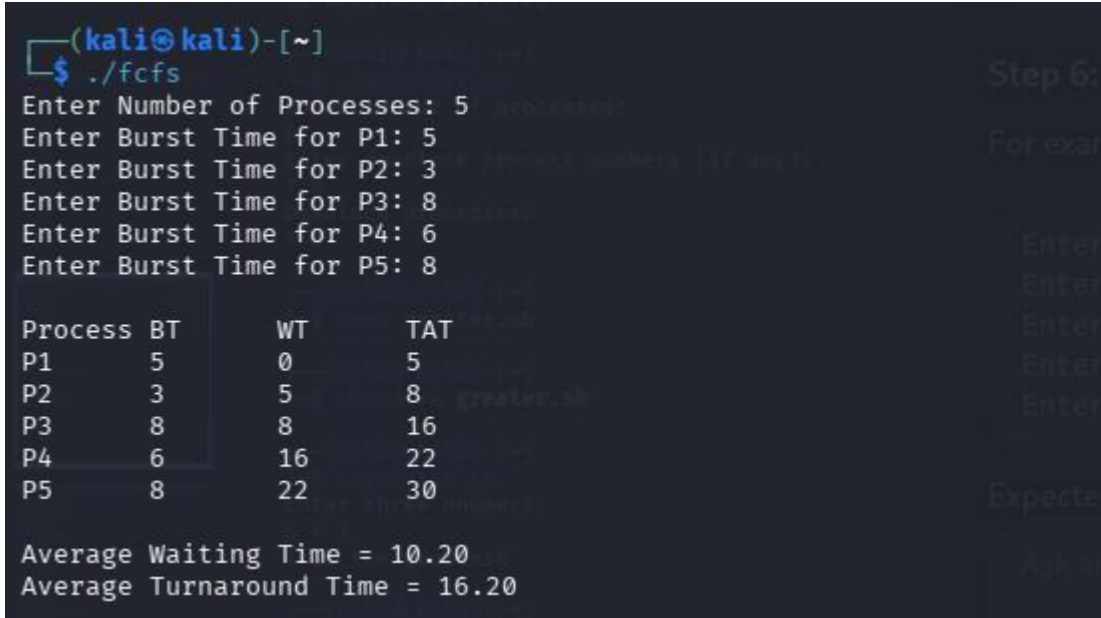
C Program

```
#include<stdio.h>

int main()
{
    int n,i;
    int bt[20],wt[20],tat[20];
    float avg_wt=0,avg_tat=0;
    printf("Enter Number of Processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("Enter Burst Time for P%d: ",i+1);
        scanf("%d",&bt[i]);
    }
    wt[0]=0;
    for(i=1;i<n;i++)
        wt[i]=wt[i-1]+bt[i-1];
    for(i=0;i<n;i++)
    {
        tat[i]=wt[i]+bt[i];
        avg_wt+=wt[i];
        avg_tat+=tat[i];
    }
    printf("\nProcess\tBT\tWT\tTAT\n");
    for(i=0;i<n;i++)
        printf("P%d\t%d\t%d\t%d\n",i+1,bt[i],wt[i],tat[i]);
    printf("\nAverage Waiting Time = %.2f",avg_wt/n);
```

```
printf("\nAverage Turnaround Time = %.2f\n",avg_tat/n);
return 0;
}
```

Output:



```
(kali@kali)-[~]
$ ./fcfs
Enter Number of Processes: 5
Enter Burst Time for P1: 5
Enter Burst Time for P2: 3
Enter Burst Time for P3: 8
Enter Burst Time for P4: 6
Enter Burst Time for P5: 8

Process BT      WT      TAT
P1      5       0       5
P2      3       5       8
P3      8       8      16
P4      6      16      22
P5      8      22      30

Average Waiting Time = 10.20
Average Turnaround Time = 16.20
```

Shell Program (fcfs.sh)

```
#!/bin/bash

echo "Enter Number of Processes"
read n

for ((i=0;i<n;i++))
do
echo "Enter Burst Time for P$((i+1))"
read bt[$i]
done

wt[0]=0

for ((i=1;i<n;i++))
do
wt[$i]=$((wt[i-1]+bt[i-1]))
done
```

```

echo
echo -e "Process\tBT\tWT\tTAT"

total_wt=0
total_tat=0

for ((i=0;i<n;i++))
do
tat[$i]=$((wt[i]+bt[i]))
total_wt=$((total_wt+wt[i]))
total_tat=$((total_tat+tat[i]))

echo -e "P$((i+1))\t${bt[i]}\t${wt[i]}\t${tat[i]}"
done

avg_wt=$(awk "BEGIN {printf \"%.2f\\n\", $total_wt/$n}")
avg_tat=$(awk "BEGIN {printf \"%.2f\\n\", $total_tat/$n}")

echo

echo "Average Waiting Time = $avg_wt"
echo "Average Turnaround Time = $avg_tat"

```

Output:

```

(kali@kali)-[~]
└─$ ./fcfs.sh
Enter Number of Processes
4
Enter Burst Time for P1
3
Enter Burst Time for P2
4
Enter Burst Time for P3
5
Enter Burst Time for P4
6
Process BT      WT      TAT
P1      3       0       3
P2      4       3       7
P3      5       7      12
P4      6      12      18

Average Waiting Time = 5.50
Average Turnaround Time = 10.00

```

2. Shortest Job First (SJF):

C Program

```
#include<stdio.h>

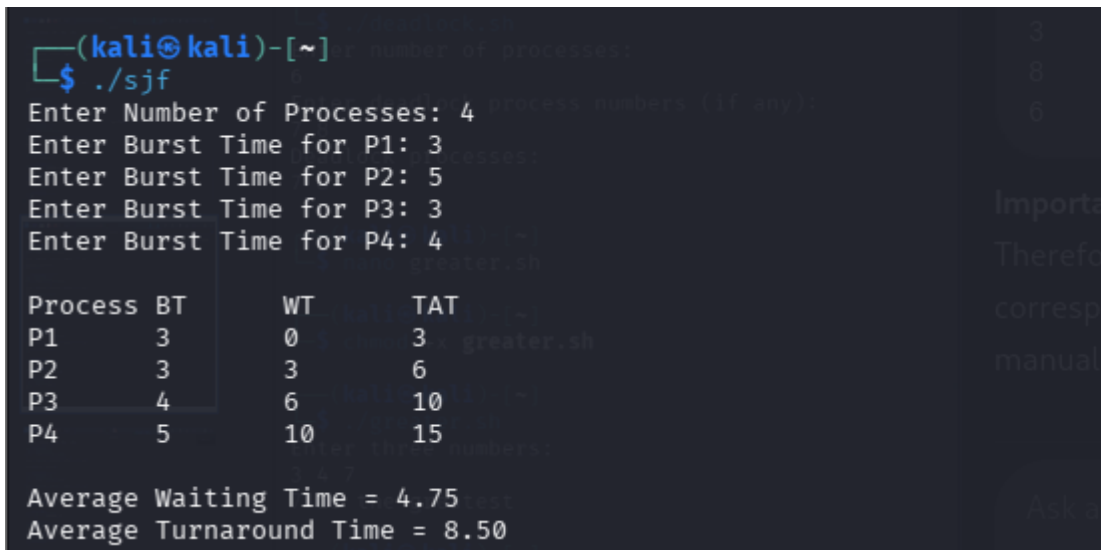
int main()
{
    int n,i,j,temp;
    int bt[20],wt[20],tat[20];
    float avg_wt=0, avg_tat=0;
    printf("Enter Number of Processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("Enter Burst Time for P%d: ",i+1);
        scanf("%d",&bt[i]);
    }
    // Sorting burst times in ascending order
    for(i=0;i<n-1;i++)
    {
        for(j=i+1;j<n;j++)
        {
            if(bt[i] > bt[j])
            {
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }
    wt[0] = 0;
```

```

for(i=1;i<n;i++)
wt[i] = wt[i-1] + bt[i-1];
printf("\nProcess\tBT\tWT\tTAT\n");
for(i=0;i<n;i++)
{
tat[i] = wt[i] + bt[i];
avg_wt += wt[i];
avg_tat += tat[i];
printf("P%d\t%d\t%d\t%d\n",i+1,bt[i],wt[i],tat[i]);
}
avg_wt = avg_wt / n;
avg_tat = avg_tat / n;
printf("\nAverage Waiting Time = %.2f", avg_wt);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
return 0;
}

```

Output:



```

(kali㉿kali)-[~]
└─$ ./sjf
Enter Number of Processes: 4
Enter Burst Time for P1: 3
Enter Burst Time for P2: 5
Enter Burst Time for P3: 3
Enter Burst Time for P4: 4

Process BT      WT      TAT
P1      3        0        3
P2      3        3        6
P3      4        6       10
P4      5       10       15

Average Waiting Time = 4.75
Average Turnaround Time = 8.50

```

Shell Program (sjf.sh)

```
#!/bin/bash
```

```
echo "Enter Number of Processes"
```

```
read n
```

```
for ((i=0;i<n;i++))
```

```
do
```

```
echo "Enter Burst Time for P$((i+1))"
```

```
read bt[$i]
```

```
done
```

```
# Sorting burst times (Ascending Order)
```

```
for ((i=0;i<n-1;i++))
```

```
do
```

```
for ((j=i+1;j<n;j++))
```

```
do
```

```
if [ ${bt[i]} -gt ${bt[j]} ]
```

```
then
```

```
temp=${bt[i]}
```

```
bt[$i]=${bt[j]}
```

```
bt[$j]=$temp
```

```
fi
```

```
done
```

```
done
```

```
wt[0]=0
```

```
echo
```

```
echo -e "Process\tBT\tWT\tTAT"
```

```
total_wt=0
```

```
total_tat=0
```

```
for ((i=0;i<n;i++))
```

```

do
if [ $i -ne 0 ]
then
wt[$i]=$((wt[i-1]+bt[i-1]))
fi

tat[$i]=$((wt[i]+bt[i]))
total_wt=$((total_wt+wt[i]))
total_tat=$((total_tat+tat[i]))

echo -e "P$((i+1))\t${bt[i]}\t${wt[i]}\t${tat[i]}"

done

avg_wt=$(awk "BEGIN {printf \"%.2f\\n\", $total_wt/$n}")
avg_tat=$(awk "BEGIN {printf \"%.2f\\n\", $total_tat/$n}")

echo

echo "Average Waiting Time = $avg_wt"
echo "Average Turnaround Time = $avg_tat"

```

Output:

```

(kali㉿kali)-[~]
└─$ chmod +x sjf.sh
└─$ ./sjf.sh
Enter Number of Processes : 3
Enter Burst Time for P1 : 2
Enter Burst Time for P2 : 4
Enter Burst Time for P3 : 5

```

Process	BT	WT	TAT
P1	2	0	2
P2	4	2	6
P3	5	6	11

```

Average Waiting Time = 2.67
Average Turnaround Time = 6.33

```

3. Priority Scheduling:

Program

```

#include<stdio.h>

int main() {
    int n,i,j,temp;
    int bt[20],pr[20],wt[20],tat[20];
    float avg_wt=0, avg_tat=0;
    printf("Enter Number of Processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("

\nEnter Burst Time for P%d: ",i+1);

        scanf("%d",&bt[i]);
        printf("Enter Priority for P%d: ",i+1);
        scanf("%d",&pr[i]);
    }
    /* Sort according to priority */
    for(i=0;i<n
-1;i++)
    {
        for(j=i+1;j<n;j++)
        {
            if(pr[i] > pr[j])
            {
                temp = pr[i];
                pr[i] = pr[j];
                pr[j] = temp;
                temp = bt[i];

```



```
bt[i] = bt[j];
```

```
bt[j] = temp;
```

```
}
```

```
}
```

```
}
```

```
wt[0] = 0;
```

```
for(i=1;i<n;i++)
```

```
wt[i] = wt[i
```

```
-1] + bt[i
```

```
-1];
```

```
printf("
```

```
\nProcess
```

```
\tPriority
```

```
\tBT
```

```
\tWT
```

```
\tTAT
```

```
\n");
```

```
for(i=0;i<n;i++)
```

```
{
```

```
tat[i] = wt[i] + bt[i];
```

```
avg_wt += wt[i];
```

```
avg_tat += tat[i];
```

```
printf("P%d\t%d\t\t%d\t%d\t%d\n",
```

```
i+1, pr[i], bt[i], wt[i], tat[i]);
```

```
}
```

```
avg_wt /= n;
```

```
avg_tat /= n;
```

```

printf("\nAverage Waiting Time = %.2f", avg_wt);

printf("\nAverage Turnaround Time = %.2f\n", avg_tat);

return 0;

}

```

Output:

```

(kali@kali)-[~]
$ ./priority
Enter Number of Processes: 3
Enter Burst Time for P1: 4
Enter Priority for P1: 3
Enter Burst Time for P2: 3
Enter Priority for P2: 2
Enter Burst Time for P3: 5
Enter Priority for P3: 4

```

Process	Priority	BT	WT	TAT
P1	2	3	0	3
P2	3	4	3	7
P3	4	5	7	12

```

Average Waiting Time = 3.33
Average Turnaround Time = 7.33

```

Shell Program (priority.sh)

```

#!/bin/bash

echo "Enter Number of Processes"

read n

for ((i=0;i<n;i++))

do

echo "Enter Burst Time for P$((i+1))"

read bt[$i]

echo "Enter Priority for P$((i+1))"

read pr[$i]

done

# Sort according to Priority

```

```

for ((i=0;i<n-1;i++))
do
for ((j=i+1;j<n;j++))
do
if [ ${pr[i]} -gt ${pr[j]} ]
then
temp=${pr[i]}
pr[$i]=${pr[j]}
pr[$j]=$temp
temp=${bt[i]}
bt[$i]=${bt[j]}
bt[$j]=$temp
fi
done
done
wt[0]=0
echo
echo -e "Process\tPriority\tBT\tWT\tTAT"
total_wt=0
total_tat=0
for ((i=0;i<n;i++))
do
if [ $i -ne 0 ]
then
wt[$i]=$((wt[i-1]+bt[i-1]))
fi
tat[$i]=$((wt[i]+bt[i]))
total_wt=$((total_wt+wt[i]))
total_tat=$((total_tat+tat[i]))

```

```

echo -e "P${(i+1)}\t${pr[i]}\t\t${bt[i]}\t${wt[i]}\t${tat[i]}"
done

avg_wt=$(awk "BEGIN {printf \"%.2f\", $total_wt/$n}")
avg_tat=$(awk "BEGIN {printf \"%.2f\", $total_tat/$n}")

echo

echo "Average Waiting Time = $avg_wt"

echo "Average Turnaround Time = $avg_tat"

```

Output:

```

(kali@kali)-[~]
$ chmod +x priority.sh
./priority.sh
Enter Number of Processes:
3
Enter Burst Time for P1:
4
Enter Priority for P1:
3
Enter Burst Time for P2:
5
Enter Priority for P2:
3
Enter Burst Time for P3:
4
Enter Priority for P3:
4

```

Process	Priority	BT	WT	TAT
P1	3	4	0	4
P2	3	4	4	8
P3	4	5	8	13

```

Average Waiting Time = 4.00
Average Turnaround Time = 8.33

```

4. Round Robin Scheduling:

C Program

```

#include<stdio.h>

int main()
{
    int n, tq, i;
    int bt[20], rem_bt[20];

```

```

int wt[20] = {0}, tat[20];
int time = 0, done;
float avg_wt = 0, avg_tat = 0;
printf("Enter Number of Processes: ");
scanf("%d", &n);
for(i = 0; i < n; i++)
{
printf("Enter Burst Time for P%d: ", i + 1);
scanf("%d", &bt[i]);
rem_bt[i] = bt[i];
}
printf("Enter Time Quantum: ");
scanf("%d", &tq);
do
{
done = 1;
for(i = 0; i < n; i++)
{
if(rem_bt[i] > 0)
{
done = 0;
if(rem_bt[i] > tq)
{
time += tq;
rem_bt[i]
-= tq;
}
}
}
else

```

```

{
time += rem_bt[i];
wt[i] = time
- bt[i];
rem_bt[i] = 0;
}
}
}
} while(!done);
printf("
\nProcess
\tBT
\tWT
\tTAT
\n");

for(i = 0; i < n; i++)
{
tat[i] = bt[i] + wt[i];

avg_wt += wt[i];
avg_tat += tat[i];
printf("P%d\t%d\t%d\t%d\n",
i + 1, bt[i], wt[i], tat[i]);
}

avg_wt /= n;
avg_tat /= n;

printf("\nAverage Waiting Time = %.2f", avg_wt);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat);

```

```
return 0;
```

```
}
```

Output:

```
(kali㉿kali)-[~]
$ ./rr
Enter Number of Processes: 3
Enter Burst Time for P1: 4
Enter Burst Time for P2: 5
Enter Burst Time for P3: 6
Enter Time Quantum: 2

Process BT    WT    TAT
P1      4      4      8
P2      5      8     13
P3      6      9     15

Average Waiting Time = 7.00
Average Turnaround Time = 12.00
```

Shell Program (rr.sh)

```
#!/bin/bash
```

```
echo "Enter Number of Processes"
```

```
read n
```

```
for ((i=0;i<n;i++))
```

```
do
```

```
echo "Enter Burst Time for P$((i+1))"
```

```
read bt[$i]
```

```
rem_bt[$i]=$(bt[$i])
```

```
done
```

```
echo "Enter Time Quantum"
```

```
read tq
```

```
time=0
```

```
while true
```

```
do
```

```
done=1
for ((i=0;i<n;i++))
do
if [ ${rem_bt[i]} -gt 0 ]
then
done=0
if [ ${rem_bt[i]} -gt $tq ]
then
time=$((time+tq))
rem_bt[$i]=$((rem_bt[i]-tq))
else
time=$((time+rem_bt[i]))
wt[$i]=$((time-bt[i]))
rem_bt[$i]=0
fi
fi
done
[ $done -eq 1 ] && break
done
echo
echo -e "Process\tBT\tWT\tTAT"
total_wt=0
total_tat=0
for ((i=0;i<n;i++))
do
tat[$i]=$((bt[i]+wt[i]))
total_wt=$((total_wt+wt[i]))
total_tat=$((total_tat+tat[i]))
echo -e "P$((i+1))\t${bt[i]}\t${wt[i]}\t${tat[i]}"
```


done

```
avg_wt=$(awk "BEGIN {printf \"%.2f\\\", $total_wt/$n}")
```

```
avg_tat=$(awk "BEGIN {printf \"%.2f\\\", $total_tat/$n}")
```

echo

```
echo "Average Waiting Time = $avg_wt"
```

```
echo "Average Turnaround Time = $avg_tat"
```

Output:

```
(kali㉿kali)-[~] r deadlock process numbers (if any):  
$ chmod +x rr.sh  
./rr.sh  
no deadlock detected  
Enter Number of Processes  
3  
Enter Burst Time for P1  
4  
Enter Burst Time for P2  
5  
Enter Burst Time for P3  
6  
Enter Time Quantum  
2  
Process BT      WT      TAT  
P1      4      4      8  
P2      5      8     13  
P3      6      9     15  
Average Waiting Time = 7.00  
Average Turnaround Time = 12.00
```